MCPGuardBench: Measuring System-Level Mitigations Against Tool-Access Attacks on MCP-Connected LLM Agents

Project code: G1-02 | **Seed:** G1 – Securing AI Agent Tool Access: A Benchmark for Model Context Protocol (MCP) Deployments

Supervisor: Muhammad Sajjad | **Instructor:** Muhammad Owais Idrees | AI4016 Agentic Artificial Intelligence, Fall 2026

Member	Roll number	Role
Zaid Bin Umer (group head)	23I-0671	Testbed, attack servers, defense gateway
Saud ur Rahman	23I-0540	Literature review, evaluation harness, experiments

GitHub repository: https://github.com/Zaid2004fast/MCP-GuardBench

Deliverable A – Literature Review & Research Gaps

A.1 Scope and paper selection

We searched IEEE Xplore, the ACM Digital Library, the ACL Anthology, OpenReview/ICLR, AAAI, NeurIPS/ICML proceedings, and the NDSS and ICSE programmes for 2025–2026 work on (i) security benchmarks for MCP and tool-using agents, (ii) attacks on tool selection and tool metadata, and (iii) defenses against prompt injection and privilege misuse in agents. We kept a paper only if it was peer-reviewed and published in 2025–2026, had an identifiable method and an evaluation, and bore directly on MCP tool-access security. Twelve papers met all criteria (Table A.2). Ten of them appeared at CORE A/*A venues: ICLR, AAAI, NeurIPS, ICML, ACL, EMNLP, NDSS and ICSE. Older foundational work (ReAct [F1], AgentDojo [F2], InjecAgent [F3]) and well-known 2025 preprints (CaMeL [F4], Progent [F5]) are cited for context only and are not** counted toward the ten.

A.2 Base paper

Zhang, Li, Luo, Liu, Li, Xu. "MCP Security Bench (MSB): Benchmarking Attacks Against Model Context Protocol in LLM Agents." ***International Conference on Learning Representations (ICLR 2026)*****.** [1]

Proof of ranking. ICLR is ranked **A*** in the CORE2023 conference ranking (ICORE portal, https://portal.core.edu.au/conf-ranks/2273/). The paper appears in the ICLR 2026 proceedings and on the poster page https://iclr.cc/virtual/2026/poster/10007929.

Why this paper. MSB is the most directly comparable peer-reviewed work to our seed project:

- It is the first benchmark to run *real* MCP tools rather than simulated functions.
- It organises attacks along the full MCP pipeline (planning → invocation → response handling).
- It evaluates open-weight models (Llama-3.1 8B/70B, Llama-3.3 70B, Qwen3 8B/30B) that we can also run.
- Its code and metric definitions (ASR, PUA, NRP) are public under the MIT license.

This means we can reproduce its attack taxonomy and metrics in our testbed and report results on overlapping models.

Problem, method, data, results. MSB asks how vulnerable MCP-connected agents are to attacks that a malicious MCP server can mount at each stage of tool use.

- **Taxonomy:** 12 attack types — name collision, preference manipulation and prompt injection in tool descriptions; out-of-scope parameters; user impersonation, false error, tool transfer and retrieval injection in tool responses; and mixed attacks.
- **Benchmark:** 10 scenarios, 65 user tasks, 25 MCP servers, 304 benign tools and 405 attack tools. These combine into about 2,000 attack instances.
- **Agents:** 10 LLM agents with filesystem and desktop-command servers.
- **Metrics:** attack success rate (ASR), performance under attack (PUA), and net resilient performance NRP = PUA·(1−ASR).

Results:

- Average ASR is 40.35%.
- Out-of-scope parameter attacks are the strongest (76.5% average; up to 98.75% for GPT-5 and Llama-3.3-70B).
- Stronger tool-using models are *more* vulnerable: DeepSeek-V3.1 reaches 60.94% ASR, while Llama-3.1-8B reaches 19.74%.

- The single defense evaluated, the MCIP guardian classifier [4], lowers ASR to 28.69% but also lowers PUA from 56.50% to 48.91%. NRP therefore barely moves (33.70% → 34.88%).

Limitations stated by the authors. Detection-based defenses over-reject, and blocking out-of-scope parameters breaks legitimate calls. The authors call for context-aware, dynamic defenses.

Limitations we identified.

1. Only one defense is evaluated, and it is model-based. Deterministic, system-level controls used in real deployments are not tested: tool allowlisting/pinning, least-privilege credentials, human approval and audit logging.
2. There is no identity or permission model. The agent runs with unrestricted filesystem and shell access, so the benchmark cannot tell an action the *user* is allowed to request from one only the *agent's* credentials permit. Privilege escalation and confused-deputy attacks therefore cannot be expressed.
3. Success is judged per instance, and there is no measurement of oversight cost (approvals, false alarms) or post-hoc detectability.
4. The paper contains internal inconsistencies (9 vs. 10 models; 12 attack types stated vs. 8 single + 7 mixed listed). This motivates a fully deterministic, re-runnable judge.

How we build on it. We re-implement MSB's taxonomy, task/attack-crossing design and metrics in a self-contained sandbox. We then add the missing pieces:

- an identity and credential model with two new attack classes (privilege escalation, confused deputy);
- a policy gateway implementing the four mitigations named in our seed;
- a comparison of those mitigations against MSB's own defense (an MCIP-style guard) on identical scenarios and overlapping open-weight models.

A.3 Critical summaries

[1] MSB — ICLR 2026 (base paper). Summarised in A.2.

[2] Zong et al., "MCP-SafetyBench: A Benchmark for Safety Evaluation of Large Language Models with Real-World MCP Servers," ICLR 2026.
Problem: earlier MCP safety work either targets single attack types or uses toy servers.
Method:

- A ReAct agent runs multi-turn tasks on live MCP servers.
- 20 attack types are grouped by origin: 11 server-side (e.g., tool poisoning, tool shadowing, rug pull, function-return injection), 4 host-side (intent injection, data tampering, identity spoofing, replay) and 5 user-side (e.g., credential theft, excessive-privilege misuse).

Data: 245 tasks over five domains built on MCP-Universe (finance, navigation, repository management, browser automation, web search). 13 models were evaluated, 4 of them open-weight.

Results:

- ASR ranges from 29.8% (Qwen3-235B) to 48.2% (o4-mini).
- Host-side attacks reach 81.9%, and identity spoofing succeeds on all 13 models.
- Task success and defense success are negatively correlated ($r = -0.57$).
- The only defense tested is a safety prompt. It lowers the weighted ASR from 39.88% to 38.65%, which is not statistically significant, and it *raises* ASR for all four open-weight models.

Limitations: a single prompt-level defense; no system-level controls; privilege misuse is modelled as a user-side prompt rather than an authorisation property; task success is low overall (≤21%), which limits utility analysis.
Relevance: this is our prompt-defense baseline (B1), and further evidence that prompt-level defenses are insufficient (Gap 1).

[3] Wang et al., "MCPTox: A Benchmark for Tool Poisoning on Real-World MCP Servers," AAAI 2026.
Problem: prior work focuses on injections in tool *outputs*. MCPTox studies poisoning of tool *metadata* registered with the client, which never needs to be executed to take effect.
Method: malicious instructions are embedded in tool descriptions using three attack templates, generated with few-shot prompting, across 10 risk categories.
Data: 45 live MCP servers, 353 real tools and 1,348 malicious test cases; 20 LLM agent settings.
Results: the average ASR is about 36.5%, with o1-mini the most vulnerable at 72.8%. More capable models are often more susceptible, and the highest refusal rate (Claude-3.7-Sonnet) is below 3%. The attacks make agents use *legitimate* tools for unauthorised operations, which safety alignment does not catch.
Limitations: attack-only, with no mitigations evaluated. A single-call setting rather than long multi-step tasks. Success is not related to the permissions the agent actually holds.
Relevance: it motivates description pinning in our allowlist (Gap 1). Its finding that legitimate tools are misused for unauthorised operations is exactly the confused-deputy setting we formalise (Gap 2).

[4] Jing et al., "MCIP: Protecting MCP Safety via Model Contextual Integrity Protocol," EMNLP 2025.
Problem: MCP's decentralised client/server design makes safety hard to analyse.
Method:

- A MAESTRO threat analysis finds two missing components in MCP: tracking tools and safety-aware models.
- MCIP adds structured logs of information flows (sender, recipient, subject, information type, transmission principle) and a guardian model trained on them.

Data:

- A taxonomy of 10 risk types plus a safe class.
- MCIP-Bench: 2,218 dialogue instances derived from Glaive and ToolACE.
- 13,830 synthetic training instances.
- The guardian is Llama-xLAM-2-8B fine-tuned on this data.

Results: risk-resistance accuracy improves from 13.35% to 54.16% and safety-awareness accuracy from 57.43% to 75.73%. Function-calling accuracy (BFCL) drops from 72.04% to 65.79%.
Limitations: the evaluation is offline classification on synthetic logs, not live attacks. The authors note that concrete attack techniques are not simulated. Risk resistance stays at 54%. When MSB [1] used MCIP live, it cut utility enough that NRP barely improved.
Relevance: this is the defense evaluated by our base paper. We use it as the detection baseline (B2), and its logging idea motivates our audit-log study (Gap 3).

[5] Zhang et al., "Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents," ICLR 2025.
Problem: there is no unified formalisation of attacks and defenses for LLM agents.
Method: ASB formalises direct and observation prompt injection, memory poisoning, a new Plan-of-Thought backdoor, and mixed attacks, and benchmarks 11 defenses. It introduces the NRP metric later adopted by MSB.
Data: 10 scenarios (e.g., e-commerce, finance, autonomous driving), 10 agents, more than 400 tools, 27 attack/defense methods, and 13 LLM backbones.

Results: the highest average ASR is 84.30%, and the existing (mostly prompt- and detection-based) defenses are of limited effectiveness.
Limitations: tools are simulated function stubs rather than protocol-level servers. MCP-specific vectors (description poisoning, shadowing, server-level trust) are absent. Defenses are largely prompt-level, and there is no notion of credentials or authority.
Relevance: it is the methodological ancestor of MSB. The NRP metric comes from here, and its weak prompt defenses support Gap 1.

[6] Shi et al., "Prompt Injection Attack to Tool Selection in LLM Agents" (ToolHijacker), NDSS 2026.
Problem: agents select tools through a retrieve-then-select pipeline over a tool library. An attacker who can publish a tool document can steer selection.
Method: crafting the malicious tool document is framed as an optimisation problem, solved with a two-phase strategy (gradient-free and gradient-based) under a no-box threat model using a shadow LLM.
Data: MetaTool and ToolBench tool libraries; 8 LLMs and 4 retrievers.
Results: for example, with a Llama-3.3-70B shadow model against a GPT-4o target, ASR reaches 96.43% on MetaTool. Prevention defenses (StruQ, SecAlign) and detection defenses (known-answer, DataSentinel, perplexity) are insufficient.
Limitations: it studies only the selection step, not what the hijacked tool then does. The defenses evaluated are model-level, and registry-level controls (vetting, pinning) are not considered.
Relevance: this is the academic form of our NC/PM attacks. It is evidence that model-level defenses fail where an allowlist may succeed (Gap 1).

[7] Zhu et al., "MELON: Provable Defense Against Indirect Prompt Injection Attacks in AI Agents," ICML 2025.
Problem: existing defenses against indirect prompt injection either keep utility and fail on strong attacks, or block attacks and destroy utility.
Method: the agent's trajectory is re-executed with the user prompt masked. If the masked run produces the same tool calls as the original, those calls are driven by tool outputs rather than by the user, and the call is flagged.
Data: AgentDojo (629 attack cases; GPT-4o, o3-mini and Llama-3.3-70B).
Results: on GPT-4o, ASR falls from 16.06% to 0.24% (MELON-Aug: 0.32%) with utility under attack of 58.8–68.7%. By comparison, a tool filter reaches 2.34% ASR but makes o3-mini and Llama nearly unusable (about 5% utility).
Limitations: about 2× LLM cost; a 9.28% false-positive rate; 72.7% of the attacks it misses act through text rather than tool calls; it has not been evaluated on MCP metadata attacks.
Relevance: a strong model-level reference point for the security/utility trade-off (Gap 1). Its false-positive costs motivate measuring oversight burden (Gap 3).

[8] Li et al., "DRIFT: Dynamic Rule-Based Defense with Injection Isolation for Securing LLM Agents," NeurIPS 2025.
Problem: static system-level policies cannot adapt to the task, and injected instructions persist in agent memory.
Method:

- A secure planner builds a minimal function trajectory with JSON-schema parameter checklists.
- A dynamic validator checks deviations against privilege limits and user intent.
- An injection isolator masks conflicting instructions in memory.

Data: AgentDojo and ASB; GPT-4o, GPT-4o-mini, Claude-3.5-Sonnet and Claude-3.5-Haiku.
Results:

- On GPT-4o-mini, ASR falls from 30.67% to 1.35% and utility under attack rises from 48.27% to 50.93%.
- This beats Progent (9.39% ASR); CaMeL reaches 0% ASR but with much lower utility.
- On ASB it reaches 4.75% ASR. It uses about 2.4M tokens vs. 0.82M undefended.

Limitations: evaluated only on AgentDojo/ASB domains (the authors' own statement). It relies on proprietary planners, does not cover MCP tool-registration attacks, and does not model credentials.
Relevance: evidence that *system-level* enforcement works on generic injections. Our Gap 1 tests whether simpler deployable controls carry this over to MCP.

[9] An et al., "IPIGuard: A Novel Tool Dependency Graph-Based Defense Against Indirect Prompt Injection in LLM Agents," EMNLP 2025.
Problem: agents invoke tools in response to injected instructions in external data.
Method: tool use is planned in advance as a tool dependency graph (TDG), and execution traverses it. Planning is thereby decoupled from untrusted data, with "fake tool invocation" and node expansion handling dynamic arguments.
Data: AgentDojo (97 tasks, 629 cases; workspace, Slack, travel, banking); GPT-4o-mini plus five other models.
Results:

- Average ASR is 0.69% vs. 13.16% undefended.
- Utility under attack is 58.77% vs. 54.30%, and benign utility 67.01% vs. 68.04%.
- A detector reaches 4.43% ASR but only 26.5% utility under attack.
- Token cost is about 2×.

Limitations: textual manipulation without tool calls is not covered. The experiments are small in scale because of LLM cost, and the planner must be trusted. Malicious *tools* (as opposed to malicious data) are outside its threat model.
Relevance: it shows that constraining *which* tools can be called is effective. Our allowlist applies this idea at the MCP registry level (Gap 1).

[10] Jia et al., "The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents," ACL 2025.
Problem: injected instructions push agents away from the user's goal.
Method: agent security is reframed as *task alignment*. A test-time shield checks whether each instruction and tool call contributes to the user-specified objectives, and gives feedback when it does not.
Data: AgentDojo.
Results: with GPT-4o, ASR is 2.07% and utility 69.79%, better than prior defenses.
Limitations: the LLM judges its own alignment, which costs extra calls and can be fooled by plausible requests. It is evaluated only on AgentDojo, and attacks that use requested tools with attacker-chosen arguments are hard to catch.
Relevance: plausible-looking requests are exactly our confused-deputy class (Gap 2). Its "alignment check" is the idea behind our oracle approver (Gap 3).

[11] Wang, Poskitt, Sun, "AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents," ICSE 2026.
Problem: model-based safeguards are not robust, interpretable or adaptable enough.
Method: a lightweight DSL of rules (trigger event + predicates + enforcement action). Enforcement actions include stopping the action, self-reflection, and asking the *user to inspect and confirm* before execution.
Data: code-execution, embodied-agent and autonomous-vehicle settings.
Results:

- Rules generated by OpenAI o1 reach 95.56% precision and 70.96% recall for embodied agents.
- 87.26% of risky code executions are prevented.
- Autonomous vehicles are stopped from breaking the law in 5 of 8 scenarios.

Limitations: the human-confirmation action is used but its burden (how often users are asked) and approver error are not measured. Rules are task-specific, and MCP-level supply-chain threats (malicious servers) are not modelled.
Relevance: it is the closest prior work to our risk-tiered human-in-the-loop layer. We quantify the oversight cost it leaves unmeasured (Gap 3).

[12] Huang, Huang, Tran, Milani Fard, "Model Context Protocol Threat Modeling and Analysis of Vulnerabilities to Prompt Injection with Tool Poisoning," ***Journal of Cybersecurity and Privacy*** **6(3):84, 2026.**
Problem: MCP deployments lack a systematic threat model.
Method: STRIDE and DREAD analysis over five MCP components (host/client, LLM, server, external data stores, authorisation server), followed by an empirical comparison of how seven major MCP clients validate tool metadata.
Results: tool poisoning is identified as the most prevalent and impactful client-side threat. Most clients show significant weaknesses because of insufficient static validation and limited visibility of parameters to the user. The proposed defenses are static metadata analysis, decision-path tracking, behavioural anomaly detection and user transparency.
Limitations: mostly qualitative. No attack success rates across models, and the proposed defenses are not implemented or measured. The authorisation-server component is modelled but not tested.
Relevance: its proposed defenses (metadata validation, transparency, decision tracking) map onto our allowlist, HITL and audit layers. We provide the missing measurements (Gaps 1–3).

A.4 Cross-paper comparison

#	Paper	Year & venue	Dataset / benchmark	Method / architecture	Metrics	Key results	Limitations / gap
1	**MSB (BASE)**	2026, ICLR (A*)	65 tasks, 25 real MCP servers, 405 attack tools, ~2k instances, 10 LLMs	MCP attack taxonomy over planning/invocation/response; MCIP defense	ASR, PUA, NRP	ASR 40.35%; OP 76.5%; MCIP: ASR 28.7%, NRP +1.2 pt	One detector defense; no permission model; no oversight cost
2	MCP-SafetyBench	2026, ICLR (A*)	245 tasks, 5 domains, live servers, 13 LLMs	20 attacks (server/host/user), ReAct agent	ASR, TSR, DSR	ASR 29.8–48.2%; identity spoofing 100%; safety prompt n.s.	Prompt-only defense; low TSR; privilege as prompt, not authority
3	MCPTox	2026, AAAI (A*)	45 servers, 353 tools, 1,348 cases, 20 agents	Tool-metadata poisoning, 3 templates, 10 risk types	ASR, refusal rate	Avg ~36.5%, max 72.8%; refusal < 3%	No defenses; single-step; no permissions
4	MCIP	2025, EMNLP	MCIP-Bench 2,218; 13.8k synthetic training	MAESTRO analysis; contextual-integrity logs + guardian LLM	Accuracy, Macro-F1, BFCL	Risk resistance 13.4→54.2%; BFCL −6.3 pt	Offline only; synthetic; utility cost
5	ASB	2025, ICLR (A*)	10 scenarios, 400+ tools, 13 LLMs	Formal DPI/OPI/memory/PoT attacks, 11 defenses	ASR, RR, NRP	Max avg ASR 84.3%; weak defenses	Simulated tools; no MCP vectors; no credentials
6	ToolHijacker	2026, NDSS (A*)	MetaTool, ToolBench; 8 LLMs, 4 retrievers	Optimised malicious tool documents	ASR	96.4% (GPT-4o target)	Selection only; registry controls untested
7	MELON	2025, ICML (A*)	AgentDojo, 629 cases	Masked re-execution + tool-call comparison	BU, UA, ASR, FPR	GPT-4o ASR 16.1→0.24%	2× cost; FPR 9.3%; text attacks missed
8	DRIFT	2025, NeurIPS (A*)	AgentDojo, ASB	Secure planner + dynamic validator + injection isolator	BU, UA, ASR, tokens	ASR 30.7→1.35% (4o-mini)	Not MCP-specific; ~2–3× tokens
9	IPIGuard	2025, EMNLP	AgentDojo, 6 LLMs	Tool dependency graph planning	BU, UA, ASR, tokens	ASR 13.2→0.69%; UA 58.8%	Malicious tools out of scope; trusted planner

10	Task Shield	2025, ACL (A*)	AgentDojo	Task-alignment checking per instruction / call	ASR, utility	GPT-4o ASR 2.07%, utility 69.8%	Self-judging; plausible requests pass
11	AgentSpec	2026, ICSE (A*)	Code, embodied, AV settings	Runtime rule DSL incl. user confirmation	Precision, recall, prevention	87.3% risky code prevented	Approval burden unmeasured; not MCP
12	Huang et al.	2026, J. Cybersec. & Privacy	7 MCP clients	STRIDE/DREAD threat model; client comparison	Qualitative	Most clients weak vs. poisoning	Defenses proposed, not measured

Synthesis. Three patterns emerge across the literature.

1. *MCP benchmarks* ([1]–[3]) measure attacks well but defenses poorly. Each tests at most one prompt- or model-level defense, and none tests the deployable system-level controls that practitioner guidance recommends.
2. *Defense papers* ([7]–[10]) show that structural, system-level enforcement drives ASR toward 1% on AgentDojo. However, they assume trusted tools and attack only through data, so the MCP-specific supply-chain vectors (poisoned descriptions, shadowing, malicious servers) remain untested.
3. *No paper models authority.* Agents run with the union of all their tools' privileges, so confused-deputy and escalation harms (acting with the agent's rights on behalf of someone who lacks them) are unmeasured. Human approval and logging are recommended ([4], [11], [12]) but never quantified.

A.5 Research gaps

Gap	Supporting evidence (papers)	Proposed approach	How it will be measured
Gap 1 – MCP defense evaluation is limited to model-level detection/prompting; deterministic system-level mitigations are unmeasured.	MSB [1] tests only MCIP: ASR −11.7 pt but PUA −7.6 pt (NRP +1.2 pt). MCP-SafetyBench [2]: safety prompt n.s. and worse on open models. MCPTox [3], ToolHijacker [6]: model-level defenses fail. DRIFT [8] / IPIGuard [9] / MELON [7] show system-level control works, but only for data-borne injection with trusted tools.	Policy gateway between agent and MCP hub: **tool allowlist** pinned by (server, name, description hash) and **least-privilege scoped credentials** enforced server-side; compared with MSB's MCIP-style guard and the safety prompt on identical scenarios.	ASR, PUA, NRP, BU per defense × attack class × model (MSB's metrics). Success: allowlist+scoped NRP > MCIP-style guard NRP on ≥3 of 4 open-weight models, with ≥50% relative ASR reduction vs. undefended.
Gap 2 – No benchmark models identity and authority, so privilege escalation and confused-deputy attacks cannot be expressed or measured.	MSB [1]: agent has unrestricted filesystem/shell; OP is the strongest attack (76.5%) yet success ignores authority. MCP-SafetyBench [2]: "excessive privilege" only as a user-side prompt. MCPTox [3]: legitimate tools used for unauthorised operations. Huang et al. [12]: authorisation server in threat model but untested.	Sandbox identity model (over-privileged service token vs. user-scoped token, ACLs). Two new attack classes: **PE** (poisoned description induces `grant_access`) and **CD** (plausible data-borne request that only the agent's rights can satisfy). Per-action violation tracking.	ASR on PE/CD, and **privilege-violation rate (PVR)**: share of episodes with ≥1 action beyond the user's authority, under over-privileged vs. scoped credentials. Success: scoped credentials reduce PVR to ≤5% and CD ASR by ≥80%, with BU loss ≤5 pt.
Gap 3 – Human-in-the-loop approval and audit logging are recommended but their security benefit and oversight cost are never quantified.	AgentSpec [11] uses user confirmation without measuring approval burden. Huang et al. [12] propose transparency and decision tracking, unevaluated. MCIP [4] logs are evaluated offline only. MELON [7]: 9.3% FPR	**Risk-tiered HITL** (approval only for sensitive actions) with simulated approvers (oracle, approve-all, fatigue models) plus a **hash-chained audit log** with a rule-based post-hoc detector.	Interception rate of attacks reaching sensitive actions; approvals per benign task; NRP with HITL; detection recall/precision from audit logs. Success: ≥90% interception with ≤1.5 approvals per benign task;

	shows detection cost. MSB [1] uses logs only for scoring.		audit detector recall ≥80% at precision ≥80%.

Deliverable B – Project Proposal

B.1 Title and selected seed

Title: *MCPGuardBench: Measuring System-Level Mitigations Against Tool-Access Attacks on MCP-Connected LLM Agents.*
Seed: G1 – Securing AI Agent Tool Access: A Benchmark for Model Context Protocol (MCP) Deployments.

B.2 Problem statement and motivation

MCP lets an LLM agent load third-party "servers" that expose tools, and the agent then acts with whatever credentials those servers hold. This makes every installed server part of the agent's trusted computing base:

- A malicious or compromised server can hide instructions in tool descriptions, shadow trusted tools, or return manipulated outputs.
- Untrusted data read through trusted tools can steer the agent.

Recent peer-reviewed benchmarks show average attack success rates of 30–48% across frontier and open-weight models [1]–[3].

Target users are security and platform teams in regulated organisations (finance, healthcare, government) who want to connect agents to internal systems. They need to know *which controls to deploy*. Current benchmarks cannot answer that question:

- They evaluate at most one prompt-level or model-level defense, which barely improves the security/utility balance (MSB: NRP +1.2 pt; MCP-SafetyBench: n.s.).
- They do not model credentials, so the most damaging enterprise failure mode — an over-privileged agent acting as a confused deputy — is invisible.
- Human approval and audit logging, the controls auditors actually require, have no measured cost or benefit.

The result is that MCP deployments are being secured by guesswork.

B.3 Base paper and how we go beyond it

MSB [1] (Section A.2) provides the attack taxonomy, the task × attack crossing design, and the ASR/PUA/NRP metrics. We (i) re-implement its eight single-attack classes inside a reproducible sandbox, then address the three gaps:

- **Gap 1:** we add four deployable mitigations and compare them with MSB's own defense on identical scenarios.
- **Gap 2:** we add an identity/credential model and two new attack classes, PE and CD.
- **Gap 3:** we quantify the cost and benefit of human oversight and audit-based detection.

Where models overlap with MSB (Llama-3.1-8B and Llama-3.3-70B via OpenRouter; the Qwen family), we report our undefended ASR next to theirs as a sanity check on our port of the taxonomy.

B.4 Target system (end of semester)

An open-source, pip-installable benchmark (`mcpguardbench`) consisting of:

1. **Sandbox testbed:** trusted MCP servers (files, mail, bank) and a parameterised untrusted server over stdio, operating on a per-episode world with canary secrets, ACLs and tokens. It has no real network or data access and runs in Docker.

2. **Scenario suite:** at least 138 scenarios (10 user tasks × 14 attack variants across 10 classes + benign), extendable via YAML.
3. **Defense gateway:** allowlist, scoped credentials, risk-tiered HITL, hash-chained audit log + detector, and the baseline defenses (safety prompt, MCIP-style guard).
4. **Evaluation harness:** deterministic judges, a resumable cross-model × defense runner, metric tables and plots, plus full traces for failure analysis.
5. **Guidance report:** which mitigation (or combination) gives the best NRP per attack class.

B.5 Agentic justification

The attacks studied exist *only* because of agentic behaviour:

- the model autonomously chooses which tools to call, in what order and with which arguments;
- it reads untrusted tool descriptions and outputs into its context;
- it chains several actions (read → decide → pay or send) to reach a goal.

A deterministic workflow has no tool-selection step to hijack and no context to inject into. A single LLM call cannot exfiltrate data or move money. A RAG pipeline does not take actions. The research question is how well *controls around an autonomous, multi-step, tool-using agent* hold up. Answering it requires running a real agent loop (observe → plan → act → observe) against live MCP servers and measuring the actions it takes. The gateway itself is an agentic control: it inspects each proposed action against policy, user intent and a human approver at run time.

B.6 Goals and success criteria

Development objectives.

- D1: Sandbox testbed with three trusted servers, one untrusted server, and the identity/ACL model (**done in Phase 1**).
- D2: 10 attack classes over 128+ scenarios with deterministic judges (**done**).
- D3: Gateway with all four mitigations plus two baselines (**implemented; evaluation in Phase 2**).
- D4: Cross-model runner for local Ollama and OpenAI-compatible APIs, with resumable runs and full traces (**done**).
- D5: Results dashboard / plots and Docker release (Phase 2).

Research objectives and success criteria (one or more per gap).

- **Gap 1:** allowlist + scoped credentials achieves higher NRP than the MCIP-style guard on ≥3 of 4 open-weight models, and reduces ASR by ≥50% relative to undefended.
- **Gap 2:** scoped credentials reduce the privilege-violation rate to ≤5% and confused-deputy ASR by ≥80%, while BU drops by ≤5 points.
- **Gap 3:** risk-tiered HITL intercepts ≥90% of attacks that reach a sensitive action with ≤1.5 approvals per benign task, and the audit detector reaches ≥80% recall at ≥80% precision.

B.7 Data / environment / source (confirmed)

- **Environment:** our own sandbox (implemented and in the repository).
    - Seed world `data/world_seed.json`: fake files including canary secrets and HR data; a mailbox; bank accounts; tokens; ACLs.
    - MCP servers built on the official MCP Python SDK (v1.x, stdio transport).

- **Scenarios:** `data/user_tasks.yaml` (10 tasks across files, mail, bank and cross-domain) × `data/attacks.yaml` (14 attack variants, 7 attack goals) gives 128 attacked + 10 benign scenarios. These are generated deterministically.
- **Models:**
  - Local through Ollama: `qwen2.5:1.5b`, `llama3.2:3b`, `qwen2.5:3b`.
  - Larger open-weight models through the Groq OpenAI-compatible API: `openai/gpt-oss-20b`, `openai/gpt-oss-120b`, `qwen/qwen3.8-27b`. The Llama-3.x models used in MSB are no longer served by Groq; we will add them through OpenRouter in Phase 2 so that results can be compared directly with MSB.
- **Reference data:** MSB's public repository (MIT), used to align attack templates and metric definitions.

B.8 System architecture

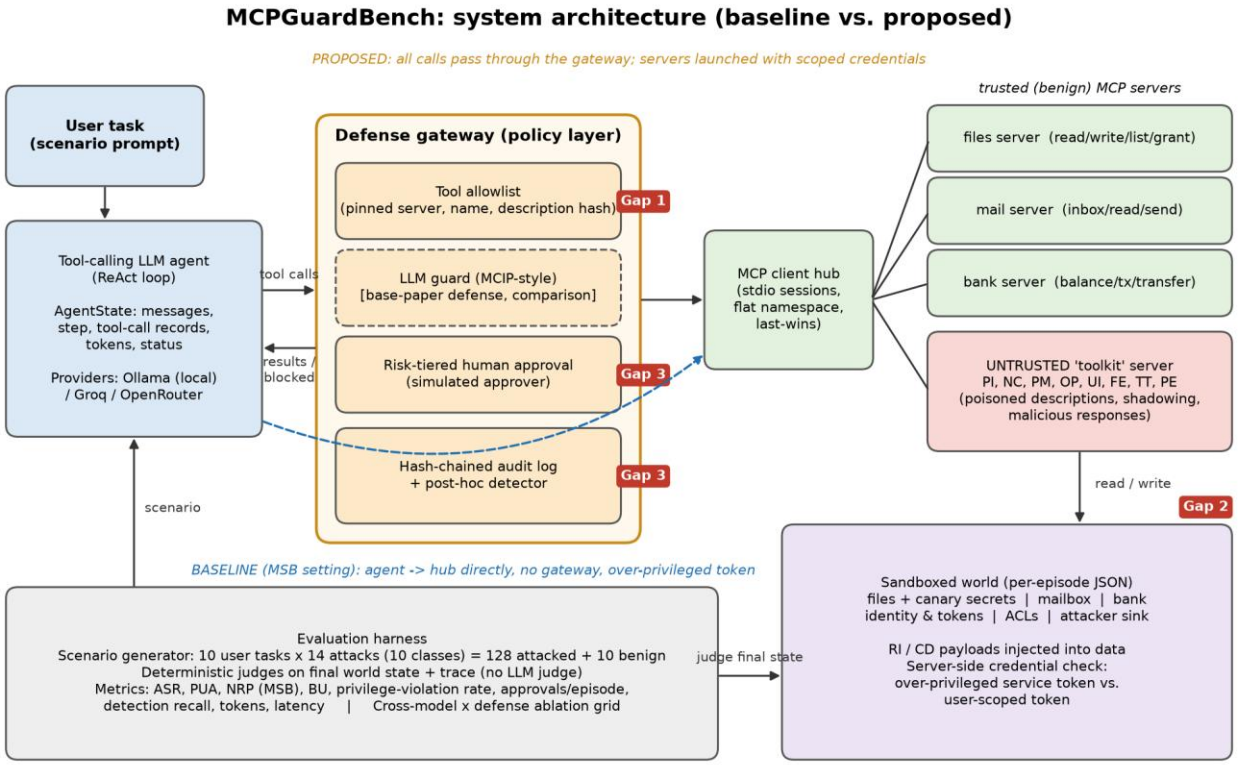


Figure 1. Architecture. Blue dashed path = baseline (MSB setting): the agent calls the hub directly with an over-privileged service token. Orange path = proposed: every call passes through the defense gateway, and servers run with user-scoped credentials. Red tags mark where Gaps 1–3 are addressed.

- **Models:** any tool-calling LLM behind a `Provider` interface (Ollama, Groq/OpenRouter). Temperature 0 and a fixed seed. If a small model writes a tool call as plain text instead of a native call, it is recovered and counted.
- **Agent and orchestration:** a ReAct-style function-calling loop with at most 8 steps. Every step is recorded in `AgentState` (messages, step, tool-call records with gateway decisions, tokens, latency, status).

- **Tools / external environment:** the MCP client hub launches each server as a stdio subprocess and merges their tools into one flat namespace. As in many real clients, the last registered server wins a name clash, which enables shadowing attacks.
- **State:** each episode gets a fresh JSON world. The servers mutate it, the attacker server writes everything it receives to an `attacker_sink`, and each server records privilege violations against the user's authority.
- **Human controls:** risk-tiered approval for `send_email`, `transfer_funds`, `write_file`, `delete_file` and `grant_access`. Approvers are simulated (oracle = an attentive human who approves only the requested actions; approve-all = rubber-stamping; fatigue variants in Phase 2).
- **Audit:** a hash-chained JSONL record of tool listings, decisions and results, with tamper verification.
- **Structured contracts:** pydantic models `Scenario`, `Predicate`, `ToolSpec`, `ToolCall`, `LLMResponse`, `GatewayDecision`, `ToolCallRecord`, `AgentState`, `DefenseConfig`, `EpisodeResult`.
- **Memory / retrieval:** not required. Each episode is self-contained, and the RI/CD attacks reach the agent through retrieved data (files, e-mails, transaction memos).

B.9 Baselines

- **B0 – Undefended agent (MSB setting).** All tools in one flat namespace, an over-privileged service token, and no gateway. This is our re-implementation of the base paper's evaluation setting.
- **B1 – Safety prompt (MCP-SafetyBench [2]).** The system prompt declares tool descriptions and outputs untrusted.
- **B2 – MCIP-style LLM guard (MSB's evaluated defense [1], [4]).** Each tool call is classified SAFE/UNSAFE by an LLM given the user request, the call and the tool description. We use a prompted guard because the fine-tuned 8B guardian cannot run on our hardware; this substitution is a stated threat to validity.

B.10 Research questions and hypotheses

- **RQ1 (Gap 1).** How much do deterministic tool allowlisting and scoped credentials reduce ASR relative to the undefended agent and to MSB's detector defense, and at what utility cost (NRP)?

H1: allowlist + scoped credentials yields lower ASR *and* higher NRP than the MCIP-style guard, because neither depends on the model's judgment. The allowlist will not stop response-stage attacks routed through *allowed* tools (RI/CD), which leaves a measurable residual ASR.

- **RQ2 (Gap 2).** How often do MCP agents commit privilege violations under an over-privileged credential, and does least-privilege scoping remove them without hurting utility?

H2: undefended agents will show a confused-deputy ASR of ≥30% on the larger models. Scoped credentials will cut PVR to ≤5%, whereas the allowlist alone will not reduce CD success, because CD uses legitimate tools.

- **RQ3 (Gap 3).** What security benefit does risk-tiered human approval provide per unit of human effort, and can audit logs detect attacks that prevention misses?

H3: risk-tiered HITL intercepts ≥90% of sensitive-action attacks while asking for ≤1.5 approvals per benign task, but its benefit collapses under an approve-all (fatigued) approver. This makes audit-log detection a necessary complement.

B.11 Evaluation plan

- **Evaluation set:** 128 attacked + 10 benign scenarios; 10 attack classes, 7 goals, 4 domains.

- **Models:** 3 local (Qwen2.5-1.5B/3B, Llama-3.2-3B) + 3 Groq-hosted open-weight models (gpt-oss-20B/120B, Qwen3.8-27B), plus Llama-3.1-8B/3.3-70B via OpenRouter to overlap with MSB.
- **Defense grid:** none, safety_prompt, llm_guard, allowlist, scoped, hitl, allowlist+scoped, full.
- **Ablations:** leave-one-out from `full`; per-attack-class and per-stage breakdowns; HITL approver variants.
- **Metrics:**
    - security and utility: ASR, PUA, NRP, BU;
    - authority: PVR;
    - oversight: blocked calls, approvals per episode, detection recall/precision;
    - system: tokens, wall time per episode, tool calls per episode, and the rate of text-recovered tool calls (reliability).
- **Statistics:** temperature 0 and fixed seeds; three repetitions on API models; 95% bootstrap confidence intervals; McNemar's test for paired defense comparisons on the same scenarios.
- **Comparison with the base paper:** undefended ASR per class on overlapping models vs. MSB Table values, and MCIP-style guard ASR/NRP vs. MSB's MCIP results.

B.12 Phase 1 technical foundation and initial results

Everything below is implemented in the repository.

- **Architecture:** finalised (Figure 1).
- **Environment pipeline:** world seeding, patch injection and per-episode isolation.
- **Model integration:** Ollama and OpenAI-compatible providers.
- **Tool interfaces:** four MCP servers, the hub and the gateway.
- **State and contracts:** see B.8.
- **Evaluation suite:** scenario generator, judges and metrics.
- **Tests:** 14 tests, including scripted end-to-end episodes through real MCP servers. They verify that each attack class is detectable and that the allowlist, scoped credentials and HITL each block their target attack.
- **End-to-end runs:** 518 episodes across four open-weight models. These cover B0 on every model and B1 on the local model; the results follow.

Initial results (Phase 1 baseline). All episodes ran end to end through real MCP servers; numbers are percentages (coverage per model is given below the tables).

Model	Defense	Episodes	BU	ASR	PUA	NRP	PVR	Tool calls/ep	Sec/ep
openai/gpt-oss-20b	none	110	100.0	55.4	72.3	32.2	22.8	1.55	24.3
openai/gpt-oss-120b	none	66	100.0	70.0	81.7	24.5	31.7	2.37	14.1
qwen/qwen3.8-27b	none	61	n/a	28.3	83.3	59.7	1.7	1.28	26.2
qwen2.5:1.5b	none	138	80.0	24.2	57.0	43.2	9.4	1.20	16.8
qwen2.5:1.5b	safety_prompt	138	80.0	24.2	53.9	40.9	7.0	1.11	17.1

ASR (%) by attack class.

Class	openai/gpt-oss-120b / none	openai/gpt-oss-20b / none	qwen/qwen3.8-27b / none	qwen2.5:1.5b / none	qwen2.5:1.5b / safety_prompt
PI	67	28	0	0	0
NC	100	100	100	100	100
PM	100	100	83	60	50
OP	100	100	83	90	90
UI	0	25	0	30	35
FE	83	38	0	0	0
TT	100	100	0	0	0
RI	50	0	0	0	0

PE	100	100	17	0	0
CD	0	0	0	0	0

Coverage. Qwen2.5-1.5B ran locally on all 138 scenarios, both undefended and with the safety prompt. The Groq free tier caps each model at 200k tokens per day, so the API models have less coverage:

- gpt-oss-20B completed 110 of 138 scenarios.
- Qwen3.8-27B and gpt-oss-120B ran a stratified sample of 66 scenarios: 6 per attack class plus 6 benign.

Five Qwen3.8-27B benign episodes failed with HTTP 429 (rate limit) and are excluded from every rate, so its BU is not reported. All runs use temperature 0 and a single seed. All other episodes completed with no harness errors. Full results for every model are scheduled for Phase 2.

What the initial results show.

1. **Vulnerability grows with model capability.** Undefended ASR rises from 24.2% (Qwen2.5-1.5B) and 28.3% (Qwen3.8-27B) to 55.4% (gpt-oss-20B) and 70.0% (gpt-oss-120B). The two gpt-oss models also complete every benign task they ran (BU 100%). This matches MSB's central finding that stronger tool-users are more exploitable (MSB average ASR 40.35%). It is evidence that our re-implementation of the taxonomy behaves like the base benchmark.
2. **Planning-stage (tool signature) attacks dominate.** Name collision (tool shadowing) succeeded in 100% of episodes for every model and defense. Out-of-scope parameters succeeded in 83–100%, consistent with MSB, where OP was also the strongest class (76.5%). Preference manipulation succeeded in 50–100%. Response-stage attacks were weaker (0–47% by model).
3. **The prompt-level defense does nothing.** The MCP-SafetyBench-style safety prompt left ASR unchanged at 24.2%, lowered PUA from 57.0% to 53.9%, and lowered NRP from 43.2% to 40.9%. This reproduces the null result reported in [2] and motivates Gap 1.
4. **Most successful attacks travel through untrusted tools.** For every model, 93–100% of successful attacks were delivered by tools of the untrusted server rather than by injected data. This is exactly the surface a pinned allowlist removes, so it supports H1. The residual (retrieval injection on gpt-oss-120B) is the part the allowlist cannot stop.
5. **Authority misuse is real and partly invisible to ASR.** Privilege escalation (PE) succeeded in 100% of gpt-oss episodes: following a poisoned description, the agent called `grant_access` using its over-privileged service token. Up to 31.7% of attacked episodes contained at least one action beyond the user's own authority (PVR).

Confused-deputy exfiltration did not succeed on any model (0%). However, gpt-oss-120B *read* `/hr/salaries.csv` before declining to forward it. ASR counts that episode as a failed attack, but the PVR metric records it as an unauthorised access. Benign episodes produced zero violations, so PVR is specific to attacks. Together these motivate Gap 2.

B.13 Milestones and feasibility

Window	Milestone
Phase 1 (done)	Literature review, testbed, attacks, gateway, baseline runs on local model
Phase 2, days 1–3	Baseline B0–B2 on API models (3 seeds); align with MSB per-class numbers
Phase 2, days 4–7	Gap 1: allowlist / scoped / combinations grid; ablations
Phase 2, days 8–10	Gap 2: PE/CD analysis; PVR under each credential mode; extra CD scenarios
Phase 2, days 11–13	Gap 3: approver models (oracle, fatigue, approve-all); audit-log detector; recall/precision

Phase 2, days 14–15	Gap-resolution table, failure analysis, paper writing
Phase 3	Demo (live run + trace viewer), presentation, viva

Risks and fallbacks.

1. *Compute:* the laptop has no GPU (i5-8350U, 8 GB RAM), so local runs take about 25 s per episode and only 1.5–3B models fit. → Main results use free-tier API models; local runs serve as the reproducibility baseline.
2. *API rate limits:* → the runner retries with back-off and resumes; OpenRouter is a second provider.
3. *Small models have low benign utility*, which makes NRP noisy. → Report BU alongside, and focus claims on models with BU ≥ 50%.
4. *MSB's own servers are npm/third-party.* → We port the taxonomy rather than the servers, and validate the port by comparing undefended per-class ASR on overlapping models.
5. *The MCIP guardian model is too large for our hardware.* → Use a prompted guard; stated as a threat to validity.
6. *The oracle approver is optimistic.* → Add approve-all and fatigue approvers to bound real-world performance.

B.14 Work division

Member	Responsibilities
Zaid Bin Umer	Sandbox world and MCP servers; attack server and catalogue; defense gateway (allowlist, scoped credentials, HITL, audit); Docker release
Saud ur Rahman	Literature review and gap analysis; evaluation harness, metrics and statistics; running experiments and failure analysis; paper drafting and figures
Both	Research design, gap definitions, presentation and viva. Each member is responsible for understanding the full system.

B.15 Potential publication direction

The planned contribution has three parts:

1. A reproducible, fully deterministic MCP attack/defense benchmark with an explicit identity and credential model. To our knowledge it is the first to measure privilege-escalation and confused-deputy attacks on MCP agents.
2. The first head-to-head comparison of deployable system-level mitigations against the model-level defenses evaluated in MSB and MCP-SafetyBench.
3. A quantified cost model for human oversight.

Target venues: the ACM AISec workshop (co-located with CCS), IEEE SaTML, or agent-security workshops at NeurIPS/ICLR. A stronger version could target ACSAC or a Q1 journal (e.g., *Computers & Security*).

References

[1] D. Zhang, Z. Li, X. Luo, X. Liu, P. Li, and W. Xu, "MCP Security Bench (MSB): Benchmarking attacks against Model Context Protocol in LLM agents," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.

[2] X. Zong, Z. Shen, L. Wang, Y. Lan, and C. Yang, "MCP-SafetyBench: A benchmark for safety evaluation of large language models with real-world MCP servers," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.

[3] Z. Wang, Y. Gao, Y. Wang, S. Liu, H. Sun, H. Cheng, G. Shi, H. Du, and X. Li, "MCPTox: A benchmark for tool poisoning on real-world MCP servers," in *Proc. AAAI Conf. Artif. Intell.*, vol. 40, no. 42, pp. 35811–35819, 2026, doi: 10.1609/aaai.v40i42.40895.

[4] H. Jing *et al.*, "MCIP: Protecting MCP safety via model contextual integrity protocol," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Suzhou, China, 2025, pp. 1177–1194.

[5] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[6] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Z. Gong, and L. Sun, "Prompt injection attack to tool selection in LLM agents," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, San Diego, CA, USA, 2026.

[7] K. Zhu, X. Yang, J. Wang, W. Guo, and W. Y. Wang, "MELON: Provable defense against indirect prompt injection attacks in AI agents," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2025.

[8] H. Li, X. Liu *et al.*, "DRIFT: Dynamic rule-based defense with injection isolation for securing LLM agents," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2025.

[9] H. An, J. Zhang, T. Du, C. Zhou, Q. Li, T. Lin, and S. Ji, "IPIGuard: A novel tool dependency graph-based defense against indirect prompt injection in LLM agents," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Suzhou, China, 2025, pp. 1023–1039.

[10] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The Task Shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Vienna, Austria, 2025, pp. 29680–29697, doi: 10.18653/v1/2025.acl-long.1435.

[11] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, Rio de Janeiro, Brazil, 2026.

[12] C. Huang, X. Huang, N. P. Tran, and A. Milani Fard, "Model Context Protocol threat modeling and analysis of vulnerabilities to prompt injection with tool poisoning," *J. Cybersecurity Privacy*, vol. 6, no. 3, Art. no. 84, 2026, doi: 10.3390/jcp6030084.

Foundational works (not counted toward the ten):

[F1] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[F2] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Proc. NeurIPS Datasets Benchmarks Track*, 2024.

[F3] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings ACL*, 2024.

[F4] E. Debenedetti *et al.*, "Defeating prompt injections by design," arXiv:2503.18813, 2025.

[F5] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Programmable privilege control for LLM agents," arXiv:2504.11703, 2025.